

Product Requirements Document

Document Q&A RAG Assistant Build Challenge

Field	Details
Document type	PRD
Challenge	Document Q&A RAG Assistant
Role	AI Engineer
Format	Take-home build + face-to-face technical walkthrough
Expected time	Up to 8 focused build hours over 3-4 elapsed days
Submission	GitHub repository + locally runnable frontend application + app documentation
Status	Draft

Context: This PRD is provided as a practical build challenge

We are not looking for a perfect production system or the most polished UI. We want to understand how you think, build, explain tradeoffs, and handle real engineering constraints.

Please choose a stack you are comfortable with. The important part is that the system demonstrates a real ingestion, indexing, retrieval, and answer-generation flow. It should not rely on hardcoded answers.

If something is incomplete, that is acceptable if you document the limitation clearly and explain what you would do next. We care about your engineering judgment as much as feature completeness.

1. Product Summary

One-liner: Document Q&A RAG Assistant is a small web application that lets a user load business-style documents, index them, ask questions, and receive grounded answers with source citations.

Problem statement: Many enterprise AI use cases require employees or customers to ask questions over messy business documents such as product guides, policy documents, FAQs, support notes, operating procedures, pricing tables, and annual reports. The challenge is not only calling an LLM. A useful application must handle loading, extraction, chunking, retrieval, citations, unsupported questions, configuration, errors, and user experience.

Solution overview: Build a working prototype where a reviewer can load or select documents, ask natural-language questions, inspect grounded answers, and see the evidence used to produce those answers.

What this challenge tests: Practical RAG engineering, frontend usability, document handling, LLM/API integration, architecture clarity, documentation, and production judgment.

Minimum viable submission

Your core submission should focus on getting this flow working well:

- Load documents.
- Index documents.
- Ask questions.
- Return grounded answers.
- Show citations or source evidence.

- Handle unsupported questions without fabricating answers.
- Document setup, architecture, assumptions, and limitations.

Online deployment, full MCP implementation, eval runners, DOCX support, and advanced extraction are optional bonus work. Do not let bonus work prevent the core local application from working.

2. Users and Review Scenarios

User	Need
Demo user	Load documents, ask questions, and inspect cited evidence without editing code.
Technical reviewer	Run the app, review the repo, inspect the architecture, and discuss implementation tradeoffs.
Interviewer	Use the submitted app and repo as the main artifact for the face-to-face walkthrough.

The application should support two review modes:

- Candidate-prepared document set: you may include a small public or synthetic enterprise-style document set, such as telecom product FAQs, troubleshooting notes, support policies, pricing tables, or operating procedures.
- Interviewer-provided unseen document set: during the interview, we may ask you to load or reason about a separate public document pack, such as annual reports or financial statements.

Use only public, synthetic, or self-created sample documents. Do not use confidential company documents or private company data.

3. Core Capabilities

#	Capability	Description	Acceptance Criteria
1	Document collections	Separate document sets so questions are answered against the intended corpus.	User can create, select, or otherwise manage a document collection and see which documents are loaded.
2	Multi-format document loading	Support common enterprise document formats.	App supports PDF, text or Markdown, and at least one structured format such as CSV or XLSX.
3	Ingestion and indexing	Convert documents into searchable retrieval units.	App extracts content, chunks or structures it, indexes it, and preserves metadata needed for citations.

#	Capability	Description	Acceptance Criteria
4	Grounded Q&A	Answer questions using retrieved document context.	User can ask a question and receive an answer grounded in retrieved evidence, not hardcoded responses.
5	Citation inspection	Make answer evidence visible.	Answer includes source citations or evidence, and user can inspect source snippets, chunks, pages, rows, or document references.
6	Unsupported-answer behavior	Avoid fabricating answers when evidence is missing.	App says when it does not have enough evidence and shows weak or partial evidence if useful.
7	API/service boundary	Keep document and retrieval operations understandable.	Repo documentation explains the boundary between frontend, ingestion, indexing, retrieval, answer generation, and source inspection.

4. Out of Scope

Exclusion	Rationale
Production-grade authentication, billing, or enterprise SSO	This is a prototype build challenge.
Perfect extraction from every PDF type	Explain limitations and tradeoffs instead.
Online deployment	Optional bonus only. A locally runnable application from GitHub is required.
Full MCP server implementation	Optional bonus only. The required expectation is a clear API/service boundary that could map to MCP.
Confidential company data	Use only public, synthetic, or self-created documents.

5. Technical Requirements

5.1 RAG pipeline

Implement a real RAG pipeline. Your approach can use frameworks or custom code.

Minimum expectations:

- Extract text or structured content from documents.
- Split content into chunks or retrieval units.
- Generate embeddings or another searchable representation.
- Store indexed content in a vector store, database, search index, or local retrieval layer.
- Retrieve relevant context for a user question.
- Generate an answer using retrieved context.
- Preserve metadata for citations.

Please explain your choices in your GitHub documentation, including chunking strategy, metadata strategy, retrieval approach, model/provider choice, and known limitations.

5.2 AI/LLM integration

You may use any reasonable model or provider, including OpenAI, Azure OpenAI, local models, or other APIs.

Minimum expectations:

- Model/provider configuration is documented.
- Required environment variables are documented.
- API keys or secrets are not committed to the repository.
- Prompting or answer-generation logic is understandable.
- Failures are handled clearly enough for a reviewer to debug.

5.3 API/service boundary expectation

The application should have a clear boundary for document and retrieval operations. This does not need to be a full MCP server, and MCP implementation should not block the core local app.

At minimum, document or implement how the system handles:

- Creating or selecting a document collection.
- Adding documents to a collection.
- Searching or retrieving relevant document content for a question.
- Returning source evidence for a cited snippet, page, row, or chunk.
- Handling errors such as unsupported files, missing collections, empty indexes, or unavailable sources.

The boundary may be API routes, backend services, functions/modules, a small tool layer, or another clear architecture. Please explain how this boundary could map to MCP tools in production if needed. A full MCP server is optional bonus work only.

6. User Experience

6.1 Primary workflow

Trigger: Reviewer opens the app and loads or selects a document collection.

Steps: Reviewer adds documents -> app indexes the content -> reviewer asks a question -> app retrieves evidence -> app generates a grounded answer -> reviewer inspects citations or source evidence.

Success state: Reviewer can tell which documents or source snippets support the answer.

6.2 Unsupported question workflow

Trigger: Reviewer asks a question not answered by the loaded documents.

Steps: App retrieves weak or no evidence -> app avoids unsupported claims -> app explains that it does not have enough information -> app shows partial evidence if useful.

Success state: Reviewer sees that the system does not fabricate an answer.

6.3 Unseen document workflow

Trigger: During the interview, the reviewer asks how the app would handle a new public document pack.

Steps: Candidate loads the document pack if feasible, or explains the ingestion and retrieval path if live loading is not practical.

Success state: Candidate can explain whether the architecture generalizes beyond the prepared sample data.

7. Required Deliverables

7.1 GitHub repository

Submit a GitHub repository containing your source code.

The repository should include:

- Frontend application code.
- Backend, API, service, or ingestion code if applicable.
- RAG pipeline code.
- API/service boundary code or documentation for document and retrieval operations.
- Sample public or synthetic documents if you include them.
- .env.example or equivalent configuration template.
- README and app documentation.

Meaningful commit history is appreciated, but the most important thing is that you can explain what you built.

7.2 Runnable frontend application

Submit a working frontend app that can run locally from the GitHub repo.

Required:

- Local run instructions.
- A frontend workflow for loading or selecting documents, asking questions, and inspecting cited evidence.
- Any required environment variables documented in the repo.

Optional bonus:

- Deployed app URL.
- Screenshots or a short demo video.

If you deploy the app, still include local setup instructions.

7.3 App documentation in GitHub

Your GitHub repo must include an app document. This can be in the README or in a separate file such as docs/app-architecture.md.

The document should explain:

- What the app does.
- How to install and run it.
- How to configure required model/API keys.
- How documents are loaded.
- How documents are chunked and indexed.
- Which embedding, search, vector store, or retrieval approach is used.
- How retrieval works.
- How answer generation works.
- How citations are produced.
- How unsupported questions are handled.
- How the API/service boundary is designed.
- Assumptions you made.
- Known limitations.
- What you would improve for production.

7.4 Application submission

Submit the assignment through the requested application process with:

- GitHub repository link.
- Frontend application link if deployed.
- Google Site or portfolio link if available.
- Facebook profile/page link if available.
- Any short notes needed to run the project.

7.5 Optional supporting materials

You may also include:

- Short demo video.
- Screenshots.
- Architecture diagram.
- Evaluation notes.
- Productionization notes.

8. Optional Bonus Features

These are not required, but they may help demonstrate deeper engineering capability:

- DOCX ingestion.
- Full MCP server implementation.
- Eval runner using a small CSV or JSON set of test questions.
- Retrieval hit or citation-quality checks.
- Reranking or hybrid search.

- Table-aware extraction.
- Multi-document comparison.
- Online deployment to a public or private environment.
- Retrieval trace in the UI or logs.
- Cost, latency, and monitoring notes.

9. Submission Checklist

Before submitting, please confirm:

- The GitHub repo is accessible.
- The app can run locally from GitHub.
- Setup instructions are clear.
- Required environment variables are documented.
- No secrets or API keys are committed.
- The frontend supports document loading and Q&A.
- Answers include citations or source evidence.
- Unsupported questions are handled safely.
- The app documentation explains architecture and tradeoffs.
- The submission includes the GitHub link and frontend link if deployed.
- Any incomplete features are documented honestly with next-step notes.

10. Face-to-face Interview Walkthrough

In the interview, we will use your submission as the main discussion artifact.

Please be ready to give a short walkthrough of:

- What you built and how to run it.
- How your RAG pipeline works from document loading to answer generation.
- How citations or source evidence are produced.
- How you handled tradeoffs, limitations, and incomplete areas.
- What you would improve next if you had more time.

We may also ask you to load or reason about an unseen public document set during the discussion, such as annual reports or financial statements, to understand how general your pipeline is.

11. Evaluation Criteria

This will be reviewed as a technical walkthrough, not as a rigid checklist. The main goal is to understand how you designed the system, what you were able to build in the timebox, and how you reason through tradeoffs.

The technical discussion will focus on:

- Whether the app can run locally from GitHub.
- How documents are loaded, extracted, chunked, and indexed.

- How retrieval works and why you chose that approach.
- How the answer is grounded in source evidence.
- How citations or source references are produced.
- How the app handles questions that are not supported by the documents.
- How the frontend supports the core workflow.
- How clearly the README or app documentation explains setup and architecture.

Bonus signals:

- Clear thinking about how this could become a production enterprise application.
- Good notes on cost, latency, monitoring, security, and data handling.
- A simple MCP/tool interface or a clear explanation of how the system could expose one.
- Experience or instincts that would help you work effectively with an enterprise AI engineering team.

12. Risks and Constraints

Risk	Mitigation
Scope becomes too large	Keep the core workflow working before adding bonus features.
UI polish hides weak retrieval	Prioritize grounding, citations, and explainable retrieval over visual polish.
Hardcoded answers	Use real ingestion and retrieval; be ready for unseen public documents in the interview.
Secrets or private data leak	Use environment variables, .env.example, and public or synthetic documents only.
Unsupported answers are fabricated	Add explicit insufficient-evidence behavior and show retrieved evidence.

13. Notes and Constraints

- Choose the stack you are most comfortable with.
- Use public, synthetic, or self-created documents only.
- Do not commit API keys or private credentials.
- Keep the project realistic for the timebox: about 4-8 focused build hours over 3-4 elapsed days.
- Prioritize a working local RAG workflow over online deployment or bonus features.
- If something is incomplete, document the limitation and explain what you would do next.